

DAIMO: An Ontology for Managing AI Models in Data Spaces

Edmundo de Elvira Mori Orrillo* Jiayun Liu

Universidad Politecnica de Madrid, Spain

Abstract. AI models are increasingly exchanged, cataloged, and consumed through data spaces, yet current documentation and catalog practices still offer weak semantic support for operational tasks such as policy-aware discovery, controlled invocation, execution traceability, and reproducible comparison of evaluation results. This paper presents DAIMO, an ontology for managing AI models in data spaces. DAIMO contributes an operational semantic model that integrates catalog, provenance, policy, and machine-learning vocabularies with explicit constructs for input-output contracts, runtime profiles, evaluation contexts, audit artifacts, and reproducibility artifacts. The ontology was developed through a requirements-driven methodology connecting conceptual design, implementation, validation, and packaging. Validation combines SHACL constraints, executable competency questions encoded as SPARQL queries, and reproducibility materials comprising notebooks, generated result tables, audit logs, and checksum-based integrity evidence. The resulting artifact shows how AI models can be published as governed assets, discovered under policy and quality restrictions, invoked through data services, traced through runs, and compared under a shared evaluation context. The paper also discusses current limits of the proposal, especially in relation to expert-based validation and richer semantic support for compatibility and benchmarking.

Keywords: ontology ; AI models ; data spaces ; semantic interoperability ; SHACL ; reproducibility

1 Introduction

Data spaces address a persistent tension in inter-organizational information sharing: organizations need to exchange assets and derive value from them without giving up

*Corresponding author: edmundo.mori.orrillo@upm.es

control over governance, access, and technical representation [1, 17, 18, 8]. When this logic is applied to AI, models become governed assets rather than isolated software components. A model that circulates in a data space may have to be cataloged, selected under policy constraints, invoked through a service, traced through execution, and compared through evaluation evidence [17, 18, 8].

Existing approaches cover parts of that lifecycle, but not the full operational chain. Documentation artifacts such as model cards, factsheets, and datasheets improve transparency [6, 7, 19]; OpenML, ModelDB, Kipoi, and TFX show how tasks, runs, protocols, dependencies, and pipelines can be operationalized [20, 21, 22, 23]; and Semantic Web vocabularies such as DCAT, ODRL, PROV-O, and ML-Schema provide strong building blocks for publication, policy, provenance, and experiment semantics [2, 4, 3, 25]. Yet these resources remain fragmented across documentation, lifecycle tooling, policy modeling, and ontology engineering traditions [15, 12, 13, 14, 16]. What is still missing is an ontology that connects those capabilities into one operational model for AI assets in data spaces.

DAIMO addresses that gap. The ontology contributes a coherent semantic framework for representing AI models as governed catalog assets linked to invocation contracts, runtime requirements, execution traces, evaluation contexts, audit evidence, and reproducibility evidence. The paper makes this contribution explicit in three ways: it defines the ontology and its reuse strategy; it shows how the model answers competency questions covering publication, discovery, execution, and evaluation; and it validates the proposal through SHACL constraints, executable SPARQL queries, and reproducibility materials that back the reported results. In that sense, the article is not only about conceptual modeling; it is also about demonstrating that the resulting ontology can support operational use in research and enterprise-style data-space settings.

The rest of the paper is structured as follows. Section 2 positions the problem with respect to related work. Section 3 describes the ontology development process. Section 4 presents the proposed ontology. Section 5 demonstrates the artifact through discovery, execution, and evaluation scenarios. Section 6 reports the validation strategy and results. Section 7 discusses implications and limitations. Section 8 summarizes availability and reproducibility. Section 9 concludes the paper.

2 Context and Related Work

The problem addressed in this paper sits at the intersection of four streams that are often treated separately: data-space architectures, model documentation and repository practices, semantic representations for machine-learning assets, and

governance vocabularies for cataloging, provenance, and policy [17, 6, 25, 2, 4, 3]. Each of these streams contributes a necessary part of the puzzle, but none of them, taken alone, is sufficient to support the end-to-end management of AI models in data spaces. What emerges from recent Semantic Web and data-space literature is not the absence of solid building blocks, but their fragmentation across different problem settings and validation traditions [8, 9, 10, 11].

The data-space perspective is important because it shifts the focus from isolated artifacts to governed exchanges among autonomous parties. Franklin et al. framed dataspace as a response to heterogeneous information environments [1], while international data spaces architecture work and later surveys make sovereignty, contract-awareness, and controlled interoperability central design assumptions [17, 18]. More recent work on federated ecosystems such as ConSolid shows how Semantic Web infrastructures have evolved toward decentralized, multi-stakeholder collaboration settings where governance and interoperability have to be engineered together [8]. In such environments, asset descriptions must support machine-actionable discovery and policy enforcement, not only human-readable documentation. This matters especially for AI models because the same resource may appear simultaneously as a catalog entry, a downloadable artifact, a service endpoint, a governed resource subject to access conditions, and an entity whose quality claims must be backed by execution and evaluation evidence.

Model documentation initiatives such as model cards and factsheets have been decisive in making AI artifacts more transparent [6, 7]. Their contribution, however, is primarily documentary: they help explain intended use, limitations, and reported performance to human readers. That makes them valuable for governance and communication, but comparatively weak as a substrate for automated discovery, filtering, and evidence linking in machine-to-machine settings. Related policy-oriented work reaches a similar boundary from the opposite direction. Pandit and Esteves show that policy vocabularies such as ODRL and DPV can make data-use conditions more explicit than textual ontology descriptions alone [15], but their focus is on health-data-sharing conditions rather than on the broader operational lifecycle of AI model assets in data spaces.

Repository and lifecycle platforms occupy an intermediate position between narrative documentation and ontology-based descriptions. OpenML demonstrates that model comparison becomes operational only when datasets, tasks, runs, and evaluation protocols are explicitly organized as related resources [20]. ModelDB and related model-lifecycle proposals make a similar point from the perspective of iterative development, versioning, and experiment tracking [21]. Kipoi and TFX show that this operational layer also includes standardized input-output formats, declared dependencies, and reproducible execution pipelines [22, 23]. These platforms solve important parts of the problem, but they are not themselves interoperable

ontology profiles for policy-aware exchange across data spaces.

Semantic resources and domain ontologies address a different layer of the problem. EXPO provides a general ontology of scientific experiments [24]; ML-Schema aims at exposing the semantics of machine-learning experiments through a reusable schema [25]; MEX offers an interchange vocabulary for describing machine-learning experiments and their results [26]; OntoDM and DMOP organize tasks, algorithms, datasets, workflows, and assumptions for data mining and semantic meta-mining [27, 28]; and provenance-oriented work on the machine-learning lifecycle emphasizes traceability, recreation, and auditability [29]. Recent SWJ examples illustrate the strength of this approach in concrete domains: RePlanIT integrates heterogeneous knowledge for digital product passports of ICT devices [9], GloSIS operationalizes a large-scale soil information model through OWL and linked data services [10], and Woods et al. connect ontology engineering with natural-language-processing workflows to improve maintenance data quality [11]. These contributions are methodologically close to DAIMO in that they combine conceptual modeling with implementation and evaluation. Their focus, however, is domain-centered rather than centered on managing AI models themselves as governed data-space assets.

Another relevant stream concerns machine-actionable validation, constraints, and publication workflows. Penteado et al. show that linked data publication methodologies often fail precisely where validation and operational quality control should be embedded into the release process [12]. Bareedu et al. derive semantic validation rules from industrial standards and demonstrate how unstructured specification documents can be transformed into formal rules [13]. Ferranti et al. formalize Wikidata property constraints through SHACL and SPARQL and make explicit the trade-offs between expressivity and operationalization [14]. Robaldo and Batsakis further show that validation in SHACL cannot always be treated independently from inference, a point that matters whenever an ontology aims to support non-trivial operational checks [16]. Together, these works make clear that executable validation is now a core expectation for ontology contributions with operational claims.

The governance layer is supported by mature vocabularies. DCAT offers a natural foundation for publication and discovery [2]; ODRL provides a well-established information model for permissions, prohibitions, and obligations [4]; and PROV-O remains a robust basis for derivation, accountability, and execution traceability [3]. The challenge, then, is not to replace these vocabularies with a monolithic ontology, but to articulate them into an ontology that can be adopted in practice and can answer operational questions about AI models in data spaces. Table 1 summarizes that gap and positions the role of DAIMO within it.

From this comparison, the research gap can be stated more precisely: there is still no ontology that links publication metadata, access policies, execution traces,

Table 1: Related work categories and the gap addressed by DAIMO.

Family	Typical strength	Limitation for data-space model management	DAIMO response
Model documentation artifacts	Rich narrative descriptions of intended use, risks, and performance	Weak machine-actionable linking across catalogs, services, policies, and evaluation artifacts	Turns minimum operational metadata into typed, queryable resources
ML lifecycle ontologies	Formal descriptions of algorithms, implementations, runs, and evaluations	Limited emphasis on publication, discovery governance, and service consumption in data spaces	Reuses lifecycle terms while adding a data-space-oriented profile
Catalog vocabularies	Discovery of assets and services through standard metadata	Do not by themselves model execution evidence or comparative evaluation	Extends publication metadata with runtime and evaluation semantics
Policy and provenance vocabularies	Strong support for access control, permissions, and lineage	Need integration with model-specific operational metadata	Connects policy, provenance, and model management in one artifact package

and comparative evaluation evidence for AI models in data spaces in a way that is executable and reproducible. Existing work provides strong precedents for domain ontologies, policy formalization, publishing methodologies, and SHACL/SPARQL-based validation [9, 10, 15, 12, 13, 14], but not yet an ontology centered on the operational lifecycle of AI model assets in a data-space setting. DAIMO is designed to fill that gap without duplicating the vocabularies that already solve parts of the problem well.

3 Ontology Development Methodology

The ontology was developed through a requirements-driven ontology-engineering process adapted to the needs of this project. Rather than following a rigid checklist, the process was organized as a sequence of connected decisions spanning requirements, conceptual design, implementation, validation, and packaging. This was necessary because the original manuscript already contained part of the concep-

tual motivation, while the technical materials had to be aligned with a defensible ontology-engineering narrative and with the validation expectations visible in recent ontology contributions [9, 10, 11, 13].

3.1 Requirements Specification

The requirements came from two complementary sources: the original manuscript and a project-specific list of requirements for managing AI models in data spaces. Together, these sources made clear that the ontology had to support four practical needs at once: models had to be publishable as trustworthy assets with persistent identity and usage metadata; participants had to be able to discover and select them according to task, domain, policy, and quality constraints; model consumption had to be operable, traceable, and auditable; and comparative evaluation had to be contextualized and backed by reproducibility evidence.

These needs were then organized from the perspective of the main stakeholders, namely model providers, model consumers, platform operators, and governance or evaluation actors. Table 2 summarizes the requirements that matter most for each of those groups and shows how they were later translated into competency questions.

Table 2: Stakeholders and representative requirements.

Stakeholder	Concern	Representative competency questions
Model provider	Publish a model as a governed asset with identity, license, policy, and invocation contract	CQ-R1, CQ-R2, CQ-R3, CQ-R5
Model consumer	Discover models that satisfy task, domain, policy, and quality restrictions	CQ-D1, CQ-D2, CQ-D4
Platform operator	Invoke services, trace runs, and reconstruct operational evidence	CQ-E1, CQ-E3, CQ-E4
Evaluator or governance actor	Compare results under a shared context and inspect reproducibility support	CQ-V1, CQ-V2, CQ-V3, CQ-V5

The full set of competency questions covers registration, discovery, execution, and evaluation concerns. Nineteen questions were identified in total, and fourteen of them were prioritized for executable validation in the current iteration. This prioritization was not arbitrary: it was used to ensure that the core claims of the paper would be backed by running artifacts rather than by narrative argument alone.

3.2 Conceptual Design and Modeling Decisions

Three design decisions shaped the ontology. The first was a reuse-first strategy. Instead of introducing a large number of new classes and properties, DAIMO reuses DCAT for publication and distribution, ODRL for usage conditions, PROV-O for derivation and activity traces, and ML Schema for implementations, runs, metrics, and evaluations. This was not only an economy-of-modeling decision, but also an interoperability decision: the more the ontology could rely on already recognized vocabularies, the easier it would be to align the artifact with existing infrastructures and expectations.

The second decision was to define DAIMO as an ontology focused on the operational lifecycle of model assets rather than as a broad ontology of AI systems. Terms were introduced when they captured relations that were not already available in the reused vocabularies and were necessary to answer prioritized competency questions. This led to a DAIMO core organized around `Model`, `IOContract`, `RuntimeProfile`, `EvaluationContext`, `AuditArtifact`, and `ReproducibilityArtifact`. The third decision followed from the first two: publication was modeled as a transversal enabling layer rather than as a standalone demonstration scenario, because discovery, invocation, and evaluation all presuppose that a model has already been cataloged with enough identity, policy, and interface metadata to become an actionable asset.

3.3 Implementation

The implementation comprises five coordinated components: an ontology specification, a SHACL constraint layer, an example knowledge graph, a competency-question query suite, and reproducibility materials. Table 3 summarizes these components and illustrates an important modeling choice of the paper: the ontology is not presented as an isolated conceptual artifact, but as the semantic core of a validated operational workflow.

The implementation process also included corrective refactoring. Namespace inconsistencies across ontology, examples, and SHACL shapes were removed, multiple-range declarations that produced undesirable inferences were simplified, and the example graph was enriched until it could support the prioritized competency questions with executable evidence. These interventions are worth stating because they show that the ontology matured not only by adding content, but also by removing brittle semantics that would have weakened validation and reuse.

Table 3: Main implementation artifacts.

Artifact component	Role in the study
Ontology specification	Defines the classes and properties used to represent governed AI model assets in data spaces
SHACL constraint layer	Formalizes structural requirements for models, services, executions, evaluation contexts, and evidence artifacts
Example knowledge graph	Instantiates the ontology with models, policies, runs, evaluations, and supporting evidence
Competency-question query suite	Tests whether the ontology can answer the operational questions that motivated the design
Reproducibility materials	Provide notebooks, generated results, audit records, and integrity evidence supporting the evaluation claims

3.4 Publication and Maintenance

The ontology uses the canonical namespace <http://purl.org/pionera/daimo#>, reflecting the intention to keep persistent identifiers separate from local deployment details. The current release already couples ontology specification, validation materials, example data, and reproducibility evidence in a form suitable for publication and external reuse. Public HTML documentation and a formal release workflow remain immediate next steps, in line with publication lifecycle recommendations that treat validation, release, and maintenance as part of the same knowledge-engineering process [12, 30].

4 The Proposed Ontology

4.1 Scope and Overview

DAIMO models AI models as governed assets that can be published, discovered, executed, and evaluated within a data-space setting. The ontology is designed to capture the operational semantics required to connect catalog publication, policy-aware selection, invocation, run traceability, and reproducibility-backed evaluation. The central modeling decision is to represent a model as a specialized catalog asset: `daimo:Model` is treated as a subclass of `dcat:Dataset`. This allows the model to inherit distribution and service publication patterns while remaining compatible with data-space catalog practices. Around that central resource, DAIMO defines classes for interface contracts, runtime requirements, evaluation contexts, audit evidence, and reproducibility evidence, thereby making explicit the relations that remain underspecified when the reused vocabularies are applied in isolation.

4.2 Core Vocabulary

Table 4 summarizes the DAIMO classes that make the profile visible.

Table 4: Core DAIMO classes.	
Class	Role in the ontology
daimo:Model	AI model published as a catalog asset and linked to policy, distribution, implementation, and evaluation evidence
daimo:IOContract	Minimum input-output contract, including input format, output format, and authentication method
daimo:RuntimeProfile	Operational requirements such as framework and hardware needs
daimo:EvaluationContext	Context shared by evaluations, including dataset, dataset version, protocol, and random seed
daimo:Artifact	Generic superclass for files or resources that support auditability or reproducibility
daimo:AuditArtifact	Artifact that helps reconstruct a run, such as a structured execution log
daimo:ReproducibilityArtifact	Artifact supporting evaluation reproducibility, such as a notebook or results table

The core DAIMO properties link these classes to the reused vocabularies and to each other. Table 5 lists the most relevant terms.

This vocabulary supports the demonstration scenario presented in this paper: three models in the same domain, runnable implementations, traced runs, shared evaluation contexts, usage policies, service endpoints, audit logs, and reproducibility artifacts. The value of the ontology lies in connecting those elements into one coherent semantic model that can support publication, discovery, execution, and comparison within a governed data-space environment.

4.3 Reuse and Alignments

DAIMO is not intended to supersede the vocabularies it reuses. Instead, it acts as an integrating ontology that aligns them for a specific operational purpose. Table 6 summarizes the reuse strategy.

This reuse strategy has two advantages. It facilitates interoperability with infrastructures that already understand DCAT catalogs, ODRL policies, provenance chains, or machine-learning experiment metadata, and it keeps the ontology aligned with established semantic practice rather than re-encoding existing knowledge models. The current version prioritizes the semantics required for publication, discovery,

Table 5: Representative DAIMO properties.

Property	Purpose
<code>daimo:hasIOContract</code>	Connects a model to its interface contract
<code>daimo:hasImplementation</code>	Connects a model to an implementation instance
<code>daimo:hasRuntimeProfile</code>	Connects an implementation to its operational requirements
<code>daimo:hasEvaluation</code>	Connects a model to an evaluation instance
<code>daimo:hasEvaluationContext</code>	Connects an evaluation to dataset, protocol, and seed information
<code>daimo:hasAuditArtifact</code>	Connects a run to supporting audit evidence
<code>daimo:hasReproducibilityArtifact</code>	Connects an evaluation to notebooks, result tables, or related artifacts
<code>daimo:task</code> and <code>daimo:applicationDomain</code>	Support functional discovery of models
<code>daimo:serviceEndpoint</code> and <code>daimo:authenticationMethod</code>	Support invocation and controlled access
<code>daimo:metricValue</code> , <code>daimo:evaluationDatasetVersion</code> , and <code>daimo:randomSeed</code>	Support contextualized and comparable evaluation evidence
<code>daimo:artifactKind</code> , <code>daimo:artifactLocation</code> , and <code>daimo:artifactChecksum</code>	Support traceable links between semantic references and materialized files

invocation, traceability, and comparative evaluation; more specialized deployment or benchmarking semantics remain natural extensions of that foundation.

5 Scenarios and Demonstration

The demonstration uses an example graph with three models in the same application domain (Climate analytics) and task (Flood risk prediction). Model0 acts as a previously published antecedent, while Model1 and Model2 are runnable and evaluated candidates. Publication metadata operates as the enabling layer of all scenarios: before a model can be discovered, invoked, or compared, it must already be identifiable, typed, distributed, and linked to policy and interface information. For

Table 6: Reuse and alignments in DAIMO.

Vocabulary	Main reused terms	Why it is reused
DCAT	dcat:Dataset, dcat:Distribution, dcat:DataService, and dcat:accessService	Standard publication and discovery model for catalog assets and services
ODRL	odrl:hasPolicy, odrl:permission, odrl:prohibition, and odrl:target	Explicit policy-aware selection and access restrictions
PROV-O	prov:wasDerivedFrom, prov:wasAssociatedWith, prov:startedAtTime, and prov:endedAtTime	Provenance, accountability, and run reconstruction
ML Schema	mls:Implementation, mls:Run, mls:Algorithm, mls:ModelEvaluation, and mls:EvaluationMeasure	Representation of implementations, executions, and evaluation semantics

that reason, the narrative of the demonstration moves from discovery to execution and then to evaluation, but all three scenarios presuppose a shared publication layer.

5.1 Discovery Scenario

The first question is whether a participant can discover and select models under operational constraints. In the example graph, all three models belong to the same task and domain, which makes them discoverable through the metadata captured by `daimo:task` and `daimo:applicationDomain`; this is what CQ-D1 demonstrates. The scenario becomes more meaningful once selection is constrained by policy and quality. CQ-D2 filters the candidate set by license and policy conditions. In the example, `Model0` and `Model1` remain eligible because they are published under an Apache-like license and linked to permissive agreements, whereas `Model2` is linked to a research-only license and a prohibitive policy pattern. In other words, DAIMO already supports a basic form of policy-aware narrowing of the discovery space without introducing a bespoke policy language.

Discovery can also be refined through evaluation evidence. CQ-D4 shows that two models share a common evaluation context and metric, but only `Model1` satisfies a threshold of 0.85 accuracy. This matters because it shows that selection in the data space does not have to depend only on descriptive metadata or textual claims: it can

also depend on explicit, queryable evidence about quality under a declared context. The same scenario also exposes a current limit of the ontology. Although DAIMO can filter models by a basic input-output contract and by contextualized quality, it does not yet provide a richer semantic model for fine-grained compatibility between model expectations and consumer-side schemas.

5.2 Execution Scenario

The execution scenario asks whether a selected model can be invoked in a controlled way and whether its use can be traced afterward. CQ-E1 retrieves service endpoints, authentication methods, and input-output contract information. In the example, all published models expose a data service, but the authentication method differs between models. This makes explicit that invocation cannot be reduced to an endpoint URL alone; it depends on interface and access metadata that may determine whether actual consumption is possible.

The operational value of the ontology becomes clearer when models, implementations, runs, and audit artifacts are queried together. CQ-E3 and CQ-E4 show that `Model1` and `Model2` each have a traced run connected to an implementation, an algorithm, an output evaluation, an associated operator, timestamps, and a structured audit log. These links allow a platform operator or evaluator to reconstruct what was executed, by whom, when, and with which resulting evaluation. That is sufficient to support a basic auditability claim: the ontology can reconstruct concrete runs through typed links and materialized evidence. It is not yet sufficient for full operational observability, because DAIMO currently captures identifiers, status, actor, timestamps, and an audit artifact reference, but not fine-grained event streams, resource-consumption measurements, or security telemetry.

5.3 Evaluation Scenario

The evaluation scenario focuses on comparability and reproducibility. CQ-V1 retrieves the evaluation context attached to each model evaluation and shows that both `Model1` and `Model2` were evaluated on `ClimateBench-v1`, version 2026.1, under the same holdout protocol and fixed seed. This is a central point of the paper, because performance values without a shared context are weak evidence for comparison. In that sense, the example follows the same basic insight that made platforms such as OpenML operationally useful for comparative experimentation: tasks, datasets, and protocols have to be part of the description, not hidden behind a single final metric [20].

Once the context is aligned, metric semantics and ranking become meaningful. CQ-V2 and CQ-V3 show that both runnable models are evaluated with the

same metric (Accuracy), but Model1 achieves 0.89 and Model2 achieves 0.82. Because the values are attached to a shared evaluation context, the ranking is not merely a numerical sort; it is a contextualized comparison. CQ-V5 then adds the reproducibility layer by showing that each evaluation is backed by a notebook and a results table, together with executed notebook copies, audit logs, and checksum-based integrity records. This makes the evaluation scenario stronger than a pure metadata example: the graph is connected to evidence that can be re-executed and independently verified.

6 Evaluation and Validation

Validation in this work is intentionally multi-layered. The ontology is not assessed only by syntactic correctness or by descriptive examples. Instead, the validation package combines parsing checks, Meta-SHACL, SHACL conformance of the example graph, competency-question execution through SPARQL, and reproducibility materials linked to the reported evaluations. This multi-layered strategy is consistent with recent SWJ ontology work in which operational claims are backed by executable rules, ontology-engineering checks, and domain-facing evaluations rather than by conceptual modeling alone [9, 11, 13, 14, 16]. For an ontology intended to support operational model management, scientific defensibility depends on whether the proposed semantics can be checked, queried, and reproduced in practice.

6.1 Validation through Competency Questions

Fourteen prioritized competency questions are implemented as SPARQL queries and executed over the example graph through an integrated validation workflow. The current suite covers four registration questions, three discovery questions, three execution questions, and four evaluation questions. Table 7 summarizes the coverage and shows that the current validation effort is distributed across the full lifecycle claimed by the paper rather than concentrated in only one scenario.

A validation workflow parses the ontology, validates the shapes graph, checks data conformance, and executes the SPARQL suite against the example graph. In the current iteration, all prioritized competency questions pass their expected minimum result thresholds, which means that the main claims made in the discovery, execution, and evaluation sections are supported by queries that actually run over the example graph. This executable orientation aligns DAIMO more closely with ontology papers that operationalize quality checks over real datasets and workflows than with papers that limit validation to illustrative examples [11, 13, 14].

Table 7: Executable competency-question validation.

Scenario block	Queries	Count	Current result
Registration and publication	CQ-R1, CQ-R2, CQ-R3, CQ-R5	4	All queries return at least the expected number of rows
Discovery and selection	CQ-D1, CQ-D2, CQ-D4	3	Discovery, policy filtering, and quality filtering succeed
Execution and auditability	CQ-E1, CQ-E3, CQ-E4	3	Service invocation, run trace, and audit metadata are retrievable
Evaluation and reproducibility	CQ-V1, CQ-V2, CQ-V3, CQ-V5	4	Context, metric semantics, ranking, and reproducibility artifacts are retrievable

6.2 SHACL Validation

SHACL provides the structural validation layer of the artifact. The current shapes cover the main node types required by the paper narrative: models, distributions, data services, input-output contracts, implementations, runtime profiles, runs, evaluations, evaluation contexts, audit artifacts, and reproducibility artifacts. In practice, the SHACL layer plays a double role. It enforces minimum completeness requirements, such as the presence of creators, licenses, service endpoints, input-output contract fields, runtime-profile metadata, evaluation-context metadata, audit-artifact metadata, and reproducibility-artifact metadata; and it ensures that the example graph is not merely illustrative, but formally constrained in a way that matches the claims made in the paper. This use of SHACL as an operational contract for the artifact is in line with current work that treats SHACL and SPARQL as complementary instruments for formalizing and checking semantics [5, 14, 16]. The validation workflow includes both Meta-SHACL, to verify the well-formedness of the shapes graph, and data validation, to verify that the example graph conforms to those shapes. Both checks pass in the current artifact package.

6.3 Structural Quality Assessment

Rather than relying on a separate external scoring framework, the current work performs a structural quality assessment centered on qualities that directly affect paper defensibility and artifact reuse. Table 8 summarizes these dimensions and makes explicit that the relevant quality criteria for this paper are not abstract ontology scores, but concrete properties such as namespace consistency, datatype clarity, traceability, and reproducibility support.

Table 8: Structural quality assessment of the current artifact.

Quality dimension	Evidence in the artifact	Interpretation
Namespace consistency	Ontology, SHACL, examples, and SPARQL suite use the same DAIMO namespace	Eliminates traceability breaks across artifacts
Datatype explicitness	Service endpoints, metric values, timestamps, random seeds, and checksums are explicitly typed	Improves validation strength and query reliability
Operational traceability	Runs link to implementations, algorithms, evaluations, actors, timestamps, and audit artifacts	Supports reconstructable execution evidence
Reproducibility support	Evaluations link to notebooks and result tables together with executed notebook copies, audit records, and checksum-based integrity evidence	Moves the paper beyond descriptive claims toward executable evidence
Integrated validation workflow	Parsing, Meta-SHACL, SHACL, and SPARQL checks are executed as one coordinated process	Strengthens repeatability and regression testing

An important by-product of this assessment was the correction of modeling issues that would otherwise have weakened the artifact. Namespace divergence across artifacts was removed, and several overly strong range declarations were simplified because they generated unintended inferences under RDFS reasoning. This reinforces a broader point: structural quality in ontology engineering is not only a matter of adding semantics, but also of avoiding semantics that make the artifact brittle or misleading.

6.4 Expert-based Validation

Expert-based validation has not yet been completed in the current iteration, and this limitation should be stated explicitly. The present work focuses on formal and executable validation because that was the most urgent gap between the original manuscript and a defensible SWJ-style ontology paper. This means that DAIMO already matches part of the validation repertoire visible in recent SWJ work, but not yet the stronger combination of formal checks and sustained stakeholder grounding found in domain-oriented ontology efforts that rely on expert input, use-case assessment, or domain-specific adoption criteria [9, 10, 11]. An expert study remains an

important next step. In the DAIMO context, such a study should involve at least a data-space catalog or governance expert, an MLOps or platform operator, and a model provider or model consumer. The purpose would not be to replace formal validation, but to assess whether the ontology captures the right abstraction level, whether the chosen profile is practical in real settings, and whether the information required by the competency questions matches the information that stakeholders can realistically produce and maintain.

7 Discussion

The main strength of DAIMO is the way it brings together publication semantics, policy semantics, execution semantics, provenance, and evaluation evidence within one ontology that can support governed model exchange. This positioning differentiates DAIMO from domain ontologies such as RePlanIT, GloSIS, or the maintenance-activity ontology, whose main effort lies in representing rich domain semantics for product passports, soil information, or industrial maintenance workflows [9, 10, 11]. DAIMO borrows from the same engineering ethos, especially the emphasis on reuse, implementation, and evaluation, but applies it to AI model assets that need to be discoverable, invocable, traceable, and comparable across organizational boundaries. That contribution is directly relevant to enterprise cataloging scenarios, governed AI services, and future research on interoperable model exchange.

That said, the ontology also has real limits, and those limits are not peripheral. The current input-output contract remains mostly syntactic rather than deeply semantic; repositories and execution frameworks such as Kipoi and TFX make clear that robust reuse also depends on explicit preprocessing, dependency, and pipeline assumptions [22, 23]. Evaluation contexts are sufficient for like-for-like comparison in the implemented example, but they do not yet capture richer benchmarking protocols, heterogeneous metric sets, or more sophisticated compatibility criteria. The provenance captured around runs is useful for reconstruction and auditability, yet it is still far from comprehensive operational observability. There is also a policy-modeling limit: although DAIMO reuses ODRL and can support policy-aware discovery, it does not yet go as far in expressing fine-grained policy semantics as work specifically dedicated to policy formalization and compatibility reasoning [15]. Likewise, while DAIMO already operationalizes SHACL and SPARQL-based validation, it does not yet engage the deeper validation-versus-inference questions that recent SHACL research has highlighted [14, 16]. Moreover, the current paper is stronger in formal validation than in human-centered validation because expert review has not yet been conducted. These limitations do not invalidate the

contribution, but they do delimit it more precisely.

This delimitation also clarifies what kind of contribution DAIMO is. The ontology should be understood as an integration layer for governed AI model management, not as a replacement for DCAT, ODRL, PROV-O, or ML Schema. Its novelty lies in the specific integration pattern required by AI model management in data spaces and in the fact that this pattern is backed by executable validation and reproducibility evidence. In that sense, DAIMO is closer to ontology contributions in which publication, quality control, and operational testing are treated as integral parts of the scientific result [12, 13, 14]. Its value depends on whether it closes a concrete operational gap with enough formal and reproducible evidence to be reused by others.

8 Availability and Reproducibility

The research artifact accompanying this paper combines the ontology, SHACL constraints, an example knowledge graph, executable competency questions, notebooks, generated result tables, audit logs, and checksum-based integrity records. This organization mirrors the claims made in the paper: ontological constructs, validation rules, example data, and reproducibility evidence are available as mutually reinforcing components rather than as disconnected supplementary material.

For the purposes of the paper, the important result is not the file layout but the reproducibility workflow it enables. The artifact supports end-to-end ontology validation, execution of the competency-question suite, generation of evaluation outputs, preservation of executed notebooks, and integrity checking of the supporting materials. As a consequence, the semantic graph does not point to hypothetical evidence; it points to evidence that is available, executable, and integrity-controlled. Reproducibility is therefore part of the scientific argument of the paper rather than an external add-on.

Public release workflow, polished HTML documentation, and persistent public hosting remain immediate next steps. This is where DAIMO still has to align with publication-oriented best practice in linked data lifecycle work, where release, validation, and maintenance are treated as explicit stages rather than as side effects of local development [12]. Related FAIR-oriented work around the machine-learning lifecycle reaches a similar conclusion from the tooling side: tracking and publication metadata have to remain connected if reuse is to be more than ad hoc file sharing [30]. The ontology namespace is already stable; the main remaining task is to complete the public-facing release process.

9 Conclusions

This paper presented DAIMO, an ontology for managing AI models in data spaces. DAIMO addresses a concrete gap between existing catalog, policy, provenance, and machine-learning vocabularies by integrating them into one semantic model for publication, discovery, execution, auditability, and evaluation with reproducibility evidence. The contribution of the paper is therefore not only the definition of classes and properties, but the demonstration that those semantics can support operational questions that matter when AI models circulate through governed data spaces.

Three results follow from that contribution. First, AI models can be represented as governed catalog assets linked to policy conditions, invocation contracts, runtime requirements, execution evidence, and contextualized evaluation results. Second, the ontology answers a meaningful set of competency questions covering model identity, policy-aware discovery, invocation metadata, run traceability, evaluation context, ranking, and reproducibility evidence. Third, the proposal is backed by an executable validation and reproducibility workflow rather than only by conceptual diagrams or narrative examples, which strengthens both its scientific defensibility and its potential for reuse in enterprise and research settings.

Future work should focus on richer semantic compatibility models for input-output contracts, stronger benchmarking and evaluation-context modeling, expert-based validation with real stakeholders, and public release packaging with full documentation. These directions extend an ontology that already provides a validated and operational foundation for governed AI model management in data spaces.

References

References

- [1] M. Franklin, A. Halevy, and D. Maier. From Databases to Dataspace: A New Abstraction for Information Management. *SIGMOD Record*, 34(4):27–33, 2005.
- [2] A. Albertoni et al. Data Catalog Vocabulary (DCAT) – Version 2. W3C Recommendation, 2020.
- [3] P. Lebo, S. Sahoo, and D. McGuinness, editors. PROV-O: The PROV Ontology. W3C Recommendation, 2013.
- [4] R. Iannella, S. Guth, and R. Steyskal, editors. ODRL Information Model 2.2. W3C Recommendation, 2018.

- [5] H. Knublauch and D. Kontokostas, editors. Shapes Constraint Language (SHACL). W3C Recommendation, 2017.
- [6] M. Mitchell et al. Model Cards for Model Reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 2019.
- [7] M. Arnold et al. FactSheets: Increasing Trust in AI Services through Supplier’s Declarations of Conformity. IBM Research Report, 2019.
- [8] J. Werbrouck et al. ConSolid: A federated ecosystem for heterogeneous multi-stakeholder projects. *Semantic Web*, 15:429–460, 2024.
- [9] A. Kurteva et al. RePlanIT Ontology for Digital Product Passports of ICT: Laptops and Data Servers. *Semantic Web*, in press.
- [10] R. Palma et al. GloSIS: The Global Soil Information System Web Ontology. *Semantic Web*, in press.
- [11] C. Woods et al. An ontology for maintenance activities and its application to data quality. *Semantic Web*, in press.
- [12] B.E. Penteado, J.C. Maldonado, and S. Isotani. Methodologies for publishing linked open government data on the Web: A systematic mapping and a unified process model. *Semantic Web*, 14:585–610, 2023.
- [13] Y.S. Bareedu et al. Deriving semantic validation rules from industrial standards: An OPC UA study. *Semantic Web*, 15:517–554, 2024.
- [14] N. Ferranti et al. Formalizing and Validating Wikidata’s Property Constraints using SHACL and SPARQL. *Semantic Web*, in press.
- [15] H.J. Pandit and B. Esteves. Enhancing Data Use Ontology (DUO) for Health-Data Sharing by Extending it with ODRL and DPV. *Semantic Web*, in press.
- [16] L. Robaldo and S. Batsakis. On the interplay between validation and inference in SHACL: an investigation on the Time Ontology. *Semantic Web*, in press.
- [17] B. Otto et al. Reference architecture model for international data spaces. *International Journal of Information Management*, 45:182–199, 2019.
- [18] C. Cappiello, A. Gal, M. Jarke, J. Rehof, and Y. Yang. Data spaces: Design, deployment, and future directions. *ACM Computing Surveys*, 55(2):Article 46, 2022.

- [19] T. Gebru et al. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.
- [20] J. Vanschoren, J.N. van Rijn, B. Bischl, and L. Torgo. OpenML: Networked science in machine learning. *SIGKDD Explorations*, 15(2):49–60, 2014.
- [21] M. Vartak et al. ModelDB: A system for machine learning model management. In *Proceedings of the Workshop on Human-in-the-Loop Data Analytics*, 2016.
- [22] Z. Avsec et al. The Kipoi repository accelerates community exchange and reuse of predictive models for genomics. *Nature Biotechnology*, 37, 2019.
- [23] D. Baylor et al. TFX: A TensorFlow-based production-scale machine learning platform. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2017.
- [24] L.N. Soldatova and R.D. King. An ontology of scientific experiments. *Journal of the Royal Society Interface*, 3(11), 2006.
- [25] G. Correa Publio et al. ML-Schema: Exposing the semantics of machine learning with schemas and ontologies. 2018.
- [26] C.M. Keet, J.D. Fernández, and P. Panov. MEX vocabulary: A lightweight interchange format for machine learning experiments. In *Proceedings of the International Semantic Web Conference*, 2016.
- [27] P. Panov, S. Džeroski, and L.N. Soldatova. OntoDM: An ontology of data mining. In *2008 IEEE International Conference on Data Mining Workshops*, 2008.
- [28] C.M. Keet et al. The data mining optimization ontology. *Web Semantics: Science, Services and Agents on the World Wide Web*, 2015.
- [29] R. Souza et al. Provenance data in the machine learning lifecycle in computational science and engineering. In *2019 IEEE/ACM Workflows in Support of Large-Scale Science*, 2019.
- [30] M. Schmidt et al. FAIRness along the machine learning lifecycle using Dataverse in combination with MLflow. In *Proceedings of the International Conference on Data Engineering Workshops*, 2021.

A Competency Questions

The competency questions guiding the ontology are grouped as follows.

Registration and publication

1. CQ-R1: What persistent identity, version, provider, and responsible actor does a published AI model have in the data space?
2. CQ-R2: What domain, task, and functional objective does the model address, and what is its input-output contract?
3. CQ-R3: Under which license, access policy, and usage conditions can the model be shared or consumed?
4. CQ-R4: How is the model published: as a downloadable artifact, as an invocable service, or through both modalities?
5. CQ-R5: What framework, software version, dependencies, and hardware requirements are needed to execute the model?
6. CQ-R6: What minimum training, validation, provenance, and evaluation evidence accompanies the model at publication time?

Discovery

1. CQ-D1: Which models satisfy a declared task, domain, or capability in the data space?
2. CQ-D2: Which models are eligible for a given consumer when license, access policy, and restrictions are considered?
3. CQ-D3: Which models are compatible with a consumer-side schema or required input-output contract?
4. CQ-D4: Which models satisfy a minimum quality threshold for a metric under a given evaluation context?

Execution

1. CQ-E1: How can a selected model be invoked, through which endpoint, authentication method, and exchange contract?
2. CQ-E2: Which operational configuration and execution resources does a model require for viable deployment or consumption?
3. CQ-E3: Which runs have been performed over a model, with which parameters, actor, purpose, time, and results?

4. CQ-E4: Which traces, logs, or audit metadata make it possible to reconstruct a concrete execution?

Evaluation and comparison

1. CQ-V1: With which dataset, dataset version, protocol, seed, and configuration was a model evaluated?
2. CQ-V2: Which metrics were used to evaluate a model, and what is the semantic definition of each metric?
3. CQ-V3: Which models achieve the best performance for a given metric under the same evaluation context?
4. CQ-V4: Which exact model variant produced a given evaluation result, and how is it related to other variants?
5. CQ-V5: Which reproducibility artifacts support an evaluation, such as notebooks, scripts, result files, or visualizations?

B SHACL Shapes

The current SHACL graph defines the main node shapes required by the paper, namely `DatasetShape`, `DistributionShape`, and `DataServiceShape` for publication and service exposure; `ModelShape` and `IOContractShape` for model identity and invocation metadata; `ImplementationShape` and `RuntimeProfileShape` for operational requirements; `RunShape` and `AuditArtifactShape` for execution traceability; and `ModelEvaluationShape`, `EvaluationContextShape`, and `ReproducibilityArtifactShape` for comparative evaluation and reproducibility support. The shapes enforce minimum completeness rather than exhaustive domain modeling. They require, among other fields, creators, licenses, distributions, service endpoints, task and domain metadata, authentication information, runtime-profile values, evaluation-context fields, timestamps, and artifact metadata.

C SPARQL Queries

The SPARQL suite currently includes fourteen executable competency questions covering registration and publication (CQ-R1, CQ-R2, CQ-R3, and CQ-R5), discovery (CQ-D1, CQ-D2, and CQ-D4), execution (CQ-E1, CQ-E3, and CQ-E4), and evaluation (CQ-V1, CQ-V2, CQ-V3, and CQ-V5). The queries are organized as a validation

suite with declared minimum expected result counts, and the validation workflow executes them together with the SHACL checks, which makes competency-question validation repeatable.

D URI Conventions

DAIMO uses the canonical namespace <http://purl.org/pionera/daimo#>, while the example graph uses the local namespace <http://pionera.org/example#> for demonstration resources. Artifact URIs in the example graph follow stable logical locations such as <https://pionera.org/repro/model1-eval.ipynb>, <https://pionera.org/repro/model1-results.csv>, and <https://pionera.org/audit/model1-run-001.jsonl>. In the reproducibility materials, these URIs are linked to materialized artifacts and checksum-based integrity records so that semantic references remain consistent with the supporting evidence.